

or your convenience, Tweepy handles the JSON for you behind the scenes, converting JSON to Python objects using classes defined in the Tweepy library.

Key Properties of a Tweet Object

A tweet (also called a *status update*) may contain a maximum of 280 characters, but the tweet objects returned by the Twitter APIs contain many **metadata** attributes that describe aspects of the tweet, such as:

- when it was created,
- who created it,
- lists of the hashtags, urls, @-mentions and media (such as images and videos, which are specified via their URLs) included in the tweet,
- and more.

The following table lists a few key attributes of a tweet object:

Attribute	Description
<code>created_at</code>	The creation date and time in UTC (Coordinated Universal Time) format.
<code>entities</code>	Twitter extracts <code>hashtags</code> , <code>urls</code> , <code>user_mentions</code> (that is, @ <i>username</i> mentions), <code>media</code> (such as images and videos), <code>symbols</code> and <code>polls</code> from tweets and places them into the <code>entities</code> dictionary as lists that you can access with these keys
<code>extended_tweet</code>	For tweets over 140 characters, contains details such as the <code>full_text</code> and <code>entities</code>
<code>favorite_count</code>	Number of times other users favorited the tweet.

<code>coordinates</code>	The coordinates (latitude and longitude) from which the tweet was sent. This is often <code>null</code> (<code>None</code> in Python) because many users disable sending location data.
<code>place</code>	Users can associate a place with a tweet. If they do, this will be a <code>Place</code> object: https://developer.twitter.com/en/docs/tweets/dictionary/overview/geo-objects#place-dictionary otherwise, it'll be <code>null</code> (<code>None</code> in Python).
<code>id</code>	The integer ID of the tweet. Twitter recommends using <code>id_str</code> for portability.
<code>id_str</code>	The string representation of the tweet's integer ID.
<code>lang</code>	Language of the tweet, such as <code>'en'</code> for English or <code>'fr'</code> for French.
<code>retweet_count</code>	Number of times other users retweeted the tweet.
<code>text</code>	The text of the tweet. If the tweet uses the new 280-character limit and contains more than 140 characters, this property will be truncated and the <code>truncated</code> property will be set to <code>true</code> . This might also occur if a 140-character tweet was retweeted and became more than 140 characters as a result.
<code>user</code>	The <code>User</code> object representing the user that posted the tweet. For <code>User</code> object JSON properties, see: https://developer.twitter.com/en/docs/tweets/dictionary/overview/user-object .

Sample Tweet JSON

Let's look at sample JSON for the following tweet from the @nasa account:

[lick here to view code image](#)

```
@NoFear1075 Great question, Anthony! Throughout its seven-year mission,
our Parker #SolarProbe spacecraft... https://t.co/xKd6ym8waT'
```

We added line numbers and reformatted some of the JSON due to wrapping. Note that some fields in Tweet JSON are not supported in every Twitter API method; such differences are explained in the online documentation for each method.

[lick here to view code image](#)

```
1 {'created_at': 'Wed Sep 05 18:19:34 +0000 2018',
2  'id': 1037404890354606082,
3  'id_str': '1037404890354606082',
4  'text': '@NoFear1075 Great question, Anthony! Throughout its seven-year
           mission, our Parker #SolarProbe spacecraft
           https://t.co/xKd6ym8waT',
5  'truncated': True,
6  'entities': {'hashtags': [{'text': 'SolarProbe', 'indices': [84, 95]}],
7               'symbols': [],
8               'user_mentions': [{'screen_name': 'NoFear1075',
9                                   'name': 'Anthony Perrone',
10                                  'id': 284339791,
11                                  'id_str': '284339791',
12                                  'indices': [0, 11]}]},
13  'urls': [{'url': 'https://t.co/xKd6ym8waT',
14               'expanded_url': 'https://twitter.com/i/web/status/
15                               1037404890354606082',
16               'display_url': 'twitter.com/i/web/status/1
17                               'indices': [117, 140]}]},
18  'source': '<a href="http://twitter.com" rel="nofollow">Twitter Web
19             Client</a>',
20  'in_reply_to_status_id': 1037390542424956928,
21  'in_reply_to_status_id_str': '1037390542424956928',
22  'in_reply_to_user_id': 284339791,
23  'in_reply_to_user_id_str': '284339791',
24  'in_reply_to_screen_name': 'NoFear1075',
25  'user': {'id': 11348282,
26            'id_str': '11348282',
27            'name': 'NASA',
28            'screen_name': 'NASA',
29            'location': '',
30            'description': 'Explore the universe and discover our home planet w
                           @NASA. We usually post in EST (UTC-5)',
31            'url': 'https://t.co/TcEE6NS8nD',
32            'entities': {'url': {'urls': [{'url': 'https://t.co/TcEE6NS8nD',
```

```
31         'expanded_url': 'http://www.nasa.gov',
32         'display_url': 'nasa.gov',
33         'indices': [[0, 23]]},
34     'description': {'urls': []}},
35     'protected': False,
36     'followers_count': 29486081,
37     'friends_count': 287,
38     'listed_count': 91928,
39     'created_at': 'Wed Dec 19 20:20:32 +0000 2007',
40     'favourites_count': 3963,
41     'time_zone': None,
42     'geo_enabled': False,
43     'verified': True,
44     'statuses_count': 53147,
45     'lang': 'en',
46     'contributors_enabled': False,
47     'is_translator': False,
48     'is_translation_enabled': False,
49     'profile_background_color': '000000',
50     'profile_background_image_url': 'http://abs.twimg.com/images/themes
        themel/bg.png',
51     'profile_background_image_url_https': 'https://abs.twimg.com/images
        themes/themel/bg.png',
52     'profile_image_url': 'http://pbs.twimg.com/profile_images/188302352
        nasalogo_twitter_normal.jpg',
53     'profile_image_url_https': 'https://pbs.twimg.com/profile_images/
        188302352/nasalogo_twitter_normal.jpg',
54     'profile_banner_url': 'https://pbs.twimg.com/profile_banners/113482
        1535145490',
55     'profile_link_color': '205BA7',
56     'profile_sidebar_border_color': '000000',
57     'profile_sidebar_fill_color': 'F3F2F2',
58     'profile_text_color': '000000',
59     'profile_use_background_image': True,
60     'has_extended_profile': True,
61     'default_profile': False,
62     'default_profile_image': False,
63     'following': True,
64     'follow_request_sent': False,
65     'notifications': False,
66     'translator_type': 'regular'},
67     'geo': None,
68     'coordinates': None,
69     'place': None,
70     'contributors': None,
71     'is_quote_status': False,
72     'retweet_count': 7,
73     'favorite_count': 19,
74     'favorited': False,
75     'retweeted': False,
76     'possibly_sensitive': False,
77     'lang': 'en'}
```

Twitter JSON Object Resources

For a complete, more readable list of the tweet object attributes, see:

<https://developer.twitter.com/en/docs/tweets/data-dictionary/overview/tweet->

or additional details that were added when Twitter moved from a limit of 140 to 280 characters per tweet, see

<https://developer.twitter.com/en/docs/tweets/data-dictionary/overview/intro-to-tweet-son.html#extendedtweet>

For a general overview of all the JSON objects that Twitter APIs return, and links to the specific object details, see

<https://developer.twitter.com/en/docs/tweets/data-dictionary/overview/intro-to-tweet-son>

12.6 TWEETPY

We'll use the Tweepy library ⁶ (<http://www.tweepy.org/>)—one of the most popular Python libraries for interacting with the Twitter APIs. Tweepy makes it easy to access Twitter's capabilities and hides from you the details of processing the JSON objects returned by the Twitter APIs. You can view Tweepy's documentation ⁷ at

⁶ Other Python libraries recommended by Twitter include Birdy, python-twitter, Python Twitter Tools, TweetPony, TwitterAPI, twitter-gobject, TwitterSearch and twython. See <https://developer.twitter.com/en/docs/developer-utilities/twitter-libraries.html> for details.

⁷ The Tweepy documentation is a work in progress. At the time of this writing, Tweepy does not have documentation for their classes corresponding to the JSON objects the Twitter APIs return. Tweepy's classes use the same attribute names and structure as the JSON objects. You can determine the correct attribute names to access by looking at Twitter's JSON documentation. We'll explain any attribute we use in our code and provide footnotes with links to the Twitter JSON descriptions.

<http://docs.tweepy.org/en/latest/>

For additional information and the Tweepy source code, visit

<https://github.com/tweepy/tweepy>

Installing Tweepy

To install Tweepy, open your Anaconda Prompt (Windows), Terminal (macOS/Linux) or shell (Linux), then execute the following command:

```
pip install tweepy==3.7
```

Windows users might need to run the Anaconda Prompt as an Administrator for proper software installation privileges. To do so, right-click Anaconda Prompt in the start menu and select **More > Run as administrator**.

Installing geopy

As you work with Tweepy, you'll also use functions from our `tweetutilities.py` file (provided with this chapter's example code). One of the utility functions in that file depends on the **geopy library** (<https://github.com/geopy/geopy>), which we'll discuss in [section 12.15](#). To install geopy, execute:

[lick here to view code image](#)

```
conda install -c conda-forge geopy
```

12.7 AUTHENTICATING WITH TWITTER VIA TWEETPY

In the next several sections, you'll invoke various cloud-based Twitter APIs via Tweepy. Here you'll begin by using Tweepy to authenticate with Twitter and create a **Tweepy API object**, which is your gateway to using the Twitter APIs over the Internet. In subsequent sections, you'll work with various Twitter APIs by invoking methods on your API object.

Before you can invoke any Twitter API, you must use your API key, API secret key, access token and access token secret to authenticate with Twitter.⁸ Launch IPython from the `ch12 examples` folder, then import the **tweepy module** and the `keys.py` file that you modified earlier in this chapter. You can import any `.py` file as a module by using the file's name *without* the `.py` extension in an `import` statement:

⁸ You may wish to create apps that enable users to log into their Twitter accounts, manage them, post tweets, read tweets from other users, search for tweets, etc. For details on user authentication see the Tweepy Authentication tutorial at

http://docs.tweepy.org/en/latest/auth_tutorial.html.

```
In [1]: import tweepy
```

```
In [2]: import keys
```

When you import `keys.py` as a module, you can individually access each of the four variables defined in that file as `keys.variable_name`.

Creating and Configuring an OAuthHandler to Authenticate with Twitter

Authenticating with Twitter via Tweepy involves two steps. First, create an object of the tweepy module's **OAuthHandler class**, passing your API key and API secret key to its constructor. A **constructor** is a function that has the same name as the class (in this case, `OAuthHandler`–) and receives the arguments used to configure the new object:

[lick here to view code image](#)

```
In [3]: auth = tweepy.OAuthHandler(keys.consumer_key,  
In [3]: auth = tweepy.OAuthHandler(keys.consumer_key,  
....:
```

Specify your access token and access token secret by calling the `OAuthHandler` object's **set_access_token method**:

[lick here to view code image](#)

```
In [4]: auth.set_access_token(keys.access_token,  
....:                        keys.access_token_secret)  
....:
```

Creating an API Object

Now, create the API object that you'll use to interact with Twitter:

[lick here to view code image](#)

```
In [5]: api = tweepy.API(auth, wait_on_rate_limit=True,  
....:                    wait_on_rate_limit_notify=True)  
....:
```

We specified three arguments in this call to the API constructor:

- `auth` is the `OAuthHandler` object containing your credentials.
- The keyword argument `wait_on_rate_limit=True` tells Tweepy to wait 15

minutes each time it reaches a given API method's rate limit. This ensures that you do not violate Twitter's rate-limit restrictions.

- The keyword argument `wait_on_rate_limit_notify=True` tells Tweepy that, if it needs to wait due to rate limits, it should notify you by displaying a message at the command line.

You're now ready to interact with Twitter via Tweepy. Note that the code examples in the next several sections are presented as a continuing IPython session, so the authorization process you went through here need not be repeated.

12.8 GETTING INFORMATION ABOUT A TWITTER ACCOUNT

After authenticating with Twitter, you can use the Tweepy API object's `get_user` method to get a `tweepy.models.User` object containing information about a user's Twitter account. Let's get a `User` object for NASA's @nasa Twitter account:

[lick here to view code image](#)

```
In [6]: nasa = api.get_user('nasa')
```

The `get_user` method calls the Twitter API's `users/show` method.⁹ Each Twitter method you call through Tweepy has a rate limit. You can call Twitter's `users/show` method up to 900 times every 15 minutes to get information on specific user accounts. As we mention other Twitter API methods, we'll provide a footnote with a link to each method's documentation in which you can view its rate limits.

⁹ <https://developer.twitter.com/en/docs/accounts-and-users/follow-search-get-users/api-reference/get-users-show>.

The `tweepy.models` classes each correspond to the JSON that Twitter returns. For example, the `User` class corresponds to a Twitter **user object**:

<https://developer.twitter.com/en/docs/tweets/data-dictionary/overview/user-object>

Each `tweepy.models` class has a method that reads the JSON and turns it into an object of the corresponding Tweepy class.

Getting Basic Account Information

Let's access some `User` object properties to display information about the `@nasa` account:

- The **`id` property** is the account ID number created by Twitter when the user joined Twitter.
- The **`name` property** is the name associated with the user's account.
- The **`screen_name` property** is the user's Twitter handle (`@nasa`). Both the `name` and `screen_name` could be created names to protect a user's privacy.
- The **`description` property** is the description from the user's profile.

[lick here to view code image](#)

```
In [7]: nasa.id
Out[7]: 11348282

In [8]: nasa.name
Out[8]: 'NASA'

In [9]: nasa.screen_name
Out[9]: 'NASA'

In [10]: nasa.description
Out[10]: 'Explore the universe and discover our home planet    with @NASA. We
```

etting the Most Recent Status Update

The `User` object's **`status` property** returns a **`twepy.models.Status`** object, which corresponds to a Twitter **tweet object**:

<https://developer.twitter.com/en/docs/tweets/data-dictionary/overview/tweet-object>

The `Status` object's **`text` property** contains the text of the account's most recent tweet:

[lick here to view code image](#)

```
In [11]: nasa.status.text
Out[11]: 'The interaction of a high-velocity young star with    the cloud of
```

he `text` property was originally for tweets up to 140 characters. The ... above indicates that the tweet text was *truncated*. When Twitter increased the limit to 280 characters,

they added an **extended_tweet property** (demonstrated later) for accessing the text and other information from tweets between 141 and 280 characters. In this case, Twitter sets `text` to a truncated version of the `extended_tweet`'s text. Also, retweeting often results in truncation because a retweet adds characters that could exceed the character limit.

Getting the Number of Followers

You can view an account's number of followers with the **followers_count property**:

[lick here to view code image](#)

```
In [12]: nasa.followers_count
Out[12]: 29453541
```

Though this number is large, there are accounts with over 100 million followers.⁹

⁹ <https://friendorfollow.com/twitter/most-followers/>.

Getting the Number of Friends

Similarly, you can view an account's number of friends (that is, the number of accounts an account follows) with the **friends_count property**:

```
In [13]: nasa.friends_count
Out[13]: 287
```

Getting Your Own Account's Information

You can use the properties in this section on your own account as well. To do so, call the Tweepy API object's **me method**, as in:

```
me = api.me()
```

This returns a `User` object for the account you used to authenticate with Twitter in the preceding section.

12.9 INTRODUCTION TO TWEETPY CURSORS: GETTING AN ACCOUNT'S FOLLOWERS AND FRIENDS

When invoking Twitter API methods, you often receive as results collections of objects, such as tweets in your Twitter timeline, tweets in another account's timeline or lists of tweets that match specified search criteria. A **timeline** consists of tweets sent by that user

and by that user's friends—that is, other accounts that the user follows.

Each Twitter API method's documentation discusses the maximum number of items the method can return in one call—this is known as a **page** of results. When you request more results than a given method can return, Twitter's JSON responses indicate that there are more pages to get. Tweepy's `Cursor` class handles these details for you. A **Cursor** invokes a specified method and checks whether Twitter indicated that there is another page of results. If so, the `Cursor` automatically calls the method again to get those results. This continues, subject to the method's rate limits, until there are no more results to process. If you configure the API object to wait when rate limits are reached (as we did), the `Cursor` will adhere to the rate limits and wait as needed between calls. The following subsections discuss `Cursor` fundamentals. For more details, see the `Cursor` tutorial at:

```
http://docs.tweepy.org/en/latest/cursor\_tutorial.html
```

12.9.1 Determining an Account's Followers

Let's use a Tweepy `Cursor` to invoke the API object's **followers method**, which calls the Twitter API's `followers/list` method¹ to obtain an account's followers. Twitter returns these in groups of 20 by default, but you can request up to 200 at a time. For demonstration purposes, we'll grab 10 of NASA's followers.

¹ <https://developer.twitter.com/en/docs/accounts-and-users/follow-search-get-users/api-reference/get-followers-list>.

Method `followers` returns `tweepy.models.User` objects containing information about each follower. Let's begin by creating a list in which we'll store the `User` objects:

```
In [14]: followers = []
```

Creating a Cursor

Next, let's create a `Cursor` object that will call the `followers` method for NASA's account, which is specified with the `screen_name` keyword argument:

[lick here to view code image](#)

```
In [15]: cursor = tweepy.Cursor(api.followers, screen_name='nasa')
```

The `Cursor`'s constructor receives as its argument the name of the method to call —`api.followers` indicates that the `Cursor` will call the `api` object's `followers`

method. If the `Cursor` constructor receives any additional keyword arguments, like `screen_name`, these will be passed to the method specified in the constructor's first argument. So, this `Cursor` specifically gets followers for the `@nasa` Twitter account.

Getting Results

Now, we can use the `Cursor` to get some followers. The following `for` statement iterates through the results of the expression `cursor.items(10)`. The `Cursor`'s **`items` method** initiates the call to `api.followers` and returns the `followers` method's results. In this case, we pass `10` to the `items` method to request only 10 results:

[lick here to view code image](#)

```
In [16]: for account in cursor.items(10):
...:     followers.append(account.screen_name)
...:
In [17]: print('Followers:',
...:         ' '.join(sorted(followers, key=lambda s: s.lower()))
...:         )
Followers: abhinavborra BHood1976 Eshwar12341 Harish90469614 heshamkisha H
```

he preceding snippet displays the followers in ascending order by calling the built-in `sorted` function. The function's second argument is the function used to determine how the elements of `followers` are sorted. In this case, we used a `lambda` that converts every follower name to lowercase letters so we can perform a case-insensitive sort.

Automatic Paging

If the number of results requested is more than can be returned by one call to `followers`, the `items` method automatically “pages” through the results by making multiple calls to `api.followers`. Recall that `followers` returns up to 20 followers at a time by default, so the preceding code needs to call `followers` only once. To get up to 200 followers at a time, we can create the `Cursor` with the `count` keyword argument, as in:

[lick here to view code image](#)

```
cursor = tweepy.Cursor(api.followers, screen_name='nasa', count=200)
```

If you do not specify an argument to the `items` method, The `Cursor` attempts to get *all* of the account's followers. For large numbers of followers, this could take a significant amount of time due to Twitter's rate limits. The Twitter API's `followers/list` method

an return a maximum of 200 followers at a time and Twitter allows a maximum of 15 calls every 15 minutes. Thus, you can only get 3000 followers every 15 minutes using Twitter's free APIs. Recall that we configured the API object to automatically wait when it hits a rate limit, so if you try to get all followers and an account has more than 3000, Tweepy will automatically pause for 15 minutes after every 3000 followers and display a message. At the time of this writing, NASA has over 29.5 million followers. At 12,000 followers per hour, it would take over 100 days to get all of NASA's followers.

Note that for this example, we could have called the `followers` method directly, rather than using a `Cursor`, since we're getting only a small number of followers. We used a `Cursor` here to show how you'll typically call `followers`. In some later examples, we'll call API methods directly to get just a few results, rather than using `Cursors`.

Getting Follower IDs Rather Than Followers

Though you can get complete `User` objects for a maximum of 200 followers at a time, you can get many more Twitter ID numbers by calling the API object's **`followers_ids method`**. This calls the Twitter API's `followers/ids` method, which returns up to 5000 ID numbers at a time (again, these rate limits could change).² You can invoke this method up to 15 times every 15 minutes, so you can get 75,000 account ID numbers per rate-limit interval. This is particularly useful when combined with the API object's **`lookup-_users method`**. This calls the Twitter API's `users/lookup` method³ which can return up to 100 `User` objects at a time and can be called up to 300 times every 15 minutes. So using this combination, you could get up to 30,000 `User` objects per rate-limit interval.

² <https://developer.twitter.com/en/docs/accounts-and-users/follow-search-get-users/api-reference/get-followers-ids>.

³ <https://developer.twitter.com/en/docs/accounts-and-users/follow-search-get-users/api-reference/get-users-lookup>.

12.9.2 Determining Whom an Account Follows

The API object's **`friends method`** calls the Twitter API's `friends/list` method⁴ to get a list of `User` objects representing an account's friends. Twitter returns these in groups of 20 by default, but you can request up to 200 at a time, just as we discussed for method `followers`. Twitter allows you to call the `friends/list` method up to 15 times every 15 minutes. Let's get 10 of NASA's friend accounts:

⁴ <https://developer.twitter.com/en/docs/accounts-and-users/follow-search-get-users/api-reference/get-users-lookup>.

[earch-get-users/api-reference/get-friends-list.](#)

[lick here to view code image](#)

```
In [18]: friends = []

In [19]: cursor = tweepy.Cursor(api.friends, screen_name='nasa')

In [20]: for friend in cursor.items(10):
...:     friends.append(friend.screen_name)
...:

In [21]: print('Friends:',
...:         ' '.join(sorted(friends, key=lambda s: s.lower())))
...:
Friends: AFSpace Astro2fish Astro_Kimiya AstroAnnimal AstroDuke NASA3DPrin
```

2.9.3 Getting a User's Recent Tweets

The API method `user_timeline` returns tweets from the timeline of a specific account. A timeline includes that account's tweets and tweets from that account's friends. The method calls the Twitter API's `statuses/user_timeline` method ⁵, which returns the most recent 20 tweets, but can return up to 200 at a time. This method can return only an account's 3200 most recent tweets. Applications using this method may call it up to 1500 times every 15 minutes.

⁵ https://developer.twitter.com/en/docs/tweets/timelines/api-reference/get-statuses-user_timeline.

Method `user_timeline` returns `Status` objects with each one representing a tweet. Each `Status`'s `user` property refers to a `tweepy.models.User` object containing information about the user who sent that tweet, such as that user's `screen_name`. A `Status`'s `text` property contains the tweet's text. Let's display the `screen_name` and `text` for three tweets from @nasa:

[lick here to view code image](#)

```
n [22]: nasa_tweets = api.user_timeline(screen_name='nasa', count=3)

In [23]: for tweet in nasa_tweets:
...:     print(f'{tweet.user.screen_name}: {tweet.text}\n')
...:
NASA: Your Gut in Space: Microorganisms in the intestinal tract play an esp
https://t.co/uLOsUhwn5p
```

```
NASA: We need your help! Want to see panels at @SXSW related to space exploration  
https://t.co/ycqMMdGKUB
```

```
NASA: "You are as good as anyone in this town, but you are no better than anyone else"  
https://t.co/nhMD4n84Nf
```

These tweets were truncated (as indicated by `...`), meaning that they probably use the newer 280-character tweet limit. We'll use the `extended_tweet` property shortly to access full text for such tweets.

In the preceding snippets, we chose to call the `user_timeline` method directly and use the `count` keyword argument to specify the number of tweets to retrieve. If you wish to get more than the maximum number of tweets per call (200), then you should use a `Cursor` to call `user_timeline` as demonstrated previously. Recall that a `Cursor` automatically pages through the results by calling the method multiple times, if necessary.

Grabbing Recent Tweets from Your Own Timeline

You can call the API method `home_timeline`, as in:

```
api.home_timeline()
```

to get tweets from *your* home timeline⁶—that is, your tweets and tweets from the people *you* follow. This method calls Twitter's `statuses/home_timeline` method.⁷ By default, `home_timeline` returns the most recent 20 tweets, but can get up to 200 at a time. Again, for more than 200 tweets from your home timeline, you should use a Tweepy `Cursor` to call `home_timeline`.

⁶Specifically for the account you used to authenticate with Twitter.

⁷https://developer.twitter.com/en/docs/tweets/timelines/api-reference/get-statuses-home_timeline.

12.10 SEARCHING RECENT TWEETS

The Tweepy API method `search` returns tweets that match a query string. According to the method's documentation, Twitter maintains its search index only for the previous seven days' tweets, and a search is not guaranteed to return all matching tweets. Method `search` calls Twitter's `search/tweets` method⁸, which returns 15 tweets at a time by default, but can return up to 100.

⁸<https://developer.twitter.com/en/docs/tweets/search/api-reference>

reference/get-search-tweets.

Utility Function `print_tweets` from `tweetutilities.py`

For this section, we created a utility function `print_tweets` that receives the results of a call to API method `search` and for each tweet displays the user's `screen_name` and the tweet's `text`. If the tweet is not in English and the `tweet.lang` is not `'und'` (undefined), we'll also translate the tweet to English using `TextBlob`, as you did in the “[Natural Language Processing \(NLP\)](#)” chapter. To use this function, import it from `tweetutilities.py`:

[click here to view code image](#)

```
In [24]: from tweetutilities import print_tweets
```

Just the `print_tweets` function's definition from that file is shown below:

[click here to view code image](#)

```
def print_tweets(tweets):
    """For each Tweepy Status object in tweets, display the
    user's screen_name and tweet text. If the language is not
    English, translate the text with TextBlob."""
    for tweet in tweets:
        print(f'{tweet.screen_name}: ', end=' ')

        if 'en' in tweet.lang:
            print(f'{tweet.text}\n')
        elif 'und' not in tweet.lang: # translate to English first
            print(f'\n ORIGINAL: {tweet.text}')
            print(f'TRANSLATED: {TextBlob(tweet.text).translate()}\n')
```

Searching for Specific Words

Let's search for three recent tweets about NASA's Mars Opportunity Rover. The search method's `q` keyword argument specifies the query string, which indicates what to search for and the `count` keyword argument specifies the number of tweets to return:

[click here to view code image](#)

```
In [25]: tweets = api.search(q='Mars Opportunity Rover', count=3)

In [26]: print_tweets(tweets)
Jacker760: NASA set a deadline on the Mars Rover opportunity! As the dust c
https://t.co/KQ7xaFgrzr
```


hivak32637174: RT @Gadgets360: NASA 'Cautiously Optimistic' of Hearing Back
<https://t.co/OliTTwRvFq>

ladyanakina: NASA's Opportunity Rover Still Silent on #Mars. <https://t.co/r>

s with other methods, if you plan to request more results than can be returned by one call to `search`, you should use a `Cursor` object.

Searching with Twitter Search Operators

You can use various Twitter search operators in your query strings to refine your search results. The following table shows several Twitter search operators. Multiple operators can be combined to construct more complex queries. To see all the operators, visit

<https://twitter.com/search-home>

and click the **operators** link.

Example	Finds tweets containing
<code>python twitter</code>	Implicit <i>logical and</i> operator—Finds tweets containing <code>python</code> <i>and</i> <code>twitter</code> .
<code>python OR twitter</code>	Logical OR operator—Finds tweets containing <code>python</code> <i>or</i> <code>twitter</code> <i>or both</i> .
<code>python ?</code>	? (question mark)—Finds tweets asking questions about <code>python</code> .
<code>planets -mars</code>	- (minus sign)—Finds tweets containing <code>planets</code> but not <code>mars</code> .
	:) (happy face)—Finds <i>positive sentiment</i> tweets

<code>python :) </code>	containing python.
<code>python :(</code>	: ((sad face)—Finds <i>negative sentiment</i> tweets containing python.
<code>since:2018-09-01</code>	Finds tweets <i>on or after</i> the specified date, which must be in the form YYYY-MM-DD.
<code>near:"New York City"</code>	Finds tweets that were sent near "New York City".
<code>from:nasa</code>	Finds tweets from the account @nasa.
<code>to:nasa</code>	Finds tweets to the account @nasa.

Let's use the `from` and `since` operators to get three tweets from NASA since September 1, 2018—you should use a date within seven days before you execute this code:

[lick here to view code image](#)

```
In [27]: tweets = api.search(q='from:nasa    since:2018-09-01', count=3)
In [28]: print_tweets(tweets)
NASA: @WYSIW Our missions    detect active burning fires, track the transport
https://t.co/jx2iUoMlIy
NASA: Scarring of the    landscape is evident in the wake of the Mendocino Co
https://t.co/Nboo5GD90m
NASA: RT @NASAGlenn: To    celebrate the #NASA60th anniversary, we're explori
```

earching for a Hashtag

Tweets often contain **hashtags** that begin with # to indicate something of importance, like a trending topic. Let's get two tweets containing the hashtag `#collegefootball`:

[lick here to view code image](#)

```
In [29]: tweets = api.search(q='#collegefootball', count=2)

In [30]: print_tweets(tweets)
dmcreek: So much for #FAU giving #OU a game. #Oklahoma #FloridaAtlantic #C
heangrychef: It's game day folks! And our BBQ game is strong. #bbq #atlan
```

2.11 SPOTTING TRENDS: TWITTER TRENDS API

If a topic “goes viral,” you could have thousands or even millions of people tweeting about it at once. Twitter refers to these as **trending topics** and maintains lists of the trending topics worldwide. Via the Twitter Trends API, you can get lists of locations with trending topics and lists of the top 50 trending topics for each location.

12.11.1 Places with Trending Topics

The Tweepy API’s **trends_available method** calls the Twitter API’s `trends/available`⁹ method to get a list of all locations for which Twitter has trending topics. Method `trends_available` returns a *list of dictionaries* representing these locations. When we executed this code, there were 467 locations with trending topics:

⁹ <https://developer.twitter.com/en/docs/trends/locations-with-trending-topics/api-reference/get-trends-available>.

[lick here to view code image](#)

```
In [31]: trends_available = api.trends_available()
In [32]: len(trends_available)
Out[32]: 467
```

The dictionary in each list element returned by `trends_available` has various information, including the location’s `name` and `woeid` (discussed below):

[lick here to view code image](#)

```
In [33]: trends_available[0]
Out[33]:
{'name': 'Worldwide',
 'placeType': {'code': 19, 'name': 'Supername'},
 'url': 'http://where.yahooapis.com/v1/place/1',
 'parentid': 0,
```

```
'country': '',  
'woeid': 1,  
'countryCode': None}
```

```
In [34]: trends_available[1]
```

```
Out[34]:
```

```
{'name': 'Winnipeg',  
 'placeType': {'code': 7, 'name': 'Town'},  
 'url': 'http://where.yahooapis.com/v1/place/2972',  
 'parentid': 23424775,  
 'country': 'Canada',  
 'woeid': 2972,  
 'countryCode': 'CA'}
```

The Twitter Trends API's `trends/place` method (discussed momentarily) uses **Yahoo! Where on Earth IDs (WOEIDs)** to look up trending topics. The WOEID 1 represents *worldwide*. Other locations have unique WOEID values greater than 1. We'll use WOEID values in the next two subsections to get worldwide trending topics and trending topics for a specific city. The following table shows WOEID values for several landmarks, cities, states and continents. Note that although these are all valid WOEIDs, Twitter does not necessarily have trending topics for all these locations.

Place	WOEID	Place	WOEID
Statue of Liberty	23617050	Iguazu Falls	468785
Los Angeles, CA	2442047	United States	23424977
Washington, D.C.	2514815	North America	24865672
Paris, France	615702	Europe	24865675

You also can search for locations close to a location that you specify with latitude and longitude values. To do so, call the Tweepy API's **`trends_closest` method**, which invokes the Twitter API's `trends/closest` method. ^o

⁰ <https://developer.twitter.com/en/docs/trends/locations-with-rendering-topics/api-reference/get-trends-closest>.

12.11.2 Getting a List of Trending Topics

The Tweepy API's **trends_place method** calls the Twitter Trends API's `trends/place` method ¹ to get the top 50 trending topics for the location with the specified WOEID. You can get the WOEIDs from the `woeid` attribute in each dictionary returned by the `trends_available` or `trends_closest` methods discussed in the previous section, or you can find a location's Yahoo! Where on Earth ID (WOEID) by searching for a city/town, state, country, address, zip code or landmark at

¹ <https://developer.twitter.com/en/docs/trends/trends-for-location/api-reference/get-trends-place>.

```
http://www.woeidlookup.com
```

You also can look up WOEID's programmatically using Yahoo!'s web services via Python libraries like `woeid` ²:

²You'll need a Yahoo! API key as described in the `woeid` modules documentation.

```
https://github.com/Ray-SunR/woeid
```

Worldwide Trending Topics

Let's get today's worldwide trending topics (your results will differ):

[lick here to view code image](#)

```
In [35]: world_trends = api.trends_place(id=1)
```

Method `trends_place` returns a one-element list containing a dictionary. The dictionary's `'trends'` key refers to a list of dictionaries representing each trend:

[lick here to view code image](#)

```
In [36]: trends_list = world_trends[0]['trends']
```

Each trend dictionary has `name`, `url`, `promoted_content` (indicating the tweet is an

advertisement), query and tweet_volume keys (shown below). The following trend is in Spanish—#BienvenidoSeptiembre means “Welcome September”:

[lick here to view code image](#)

```
In [37]: trends_list[0]
Out[37]:
{'name': '#BienvenidoSeptiembre',
 'url': 'http://twitter.com/search?q=%23BienvenidoSeptiembre',
 'promoted_content': None,
 'query': '%23BienvenidoSeptiembre',
 'tweet_volume': 15186}
```

For trends with more than 10,000 tweets, the tweet_volume is the number of tweets; otherwise, it's None. Let's use a list comprehension to filter the list so that it contains only trends with more than 10,000 tweets:

[lick here to view code image](#)

```
In [38]: trends_list = [t for t in trends_list if t['tweet_volume']]
```

Next, let's sort the trends in *descending* order by tweet_volume:

[lick here to view code image](#)

```
In [39]: from operator import itemgetter

In [40]: trends_list.sort(key=itemgetter('tweet_volume'), reverse=True)
```

Now, let's display the names of the top five trending topics:

[lick here to view code image](#)

```
In [41]: for trend in trends_list[:5]:
...:     print(trend['name'])
...:
#HBDJanaSenaniPawanKalyan
#BackToHogwarts
Khalil Mack
#ItalianGP
Alisson
```

New York City Trending Topics

Now, let's get the top five trending topics for New York City (WOEID 2459115). The following code performs the same tasks as above, but for the different WOEID:

[lick here to view code image](#)

```
In [42]: nyc_trends = api.trends_place(id=2459115) # New York City WOEID

In [43]: nyc_list = nyc_trends[0]['trends']

In [44]: nyc_list = [t for t in nyc_list if t['tweet_volume']]

In [45]: nyc_list.sort(key=itemgetter('tweet_volume'), reverse=True)

In [46]: for trend in nyc_list[:5]:
...:     print(trend['name'])
...:
#IDOL100M
#TuesdayThoughts
#HappyBirthdayLiam
NAFTA
#USOpen
```

12.11.3 Create a Word Cloud from Trending Topics

In the “Natural Language Processing” chapter, we used the WordCloud library to create word clouds. Let's use it again here, to visualize New York City's trending topics that have more than 10,000 tweets each. First, let's create a dictionary of key–value pairs consisting of the trending topic names and `tweet_volumes`:

[lick here to view code image](#)

```
In [47]: topics = {}

In [48]: for trend in nyc_list:
...:     topics[trend['name']] = trend['tweet_volume']
...:
```

Next, let's create a WordCloud from the `topics` dictionary's key–value pairs, then output the word cloud to the image file `TrendingTwitter.png` (shown after the code). The argument `prefer_horizontal=0.5` *suggests* that 50% of the words should be horizontal, though the software may ignore that to fit the content:

[lick here to view code image](#)

```
In [49]: from wordcloud import WordCloud
```

```

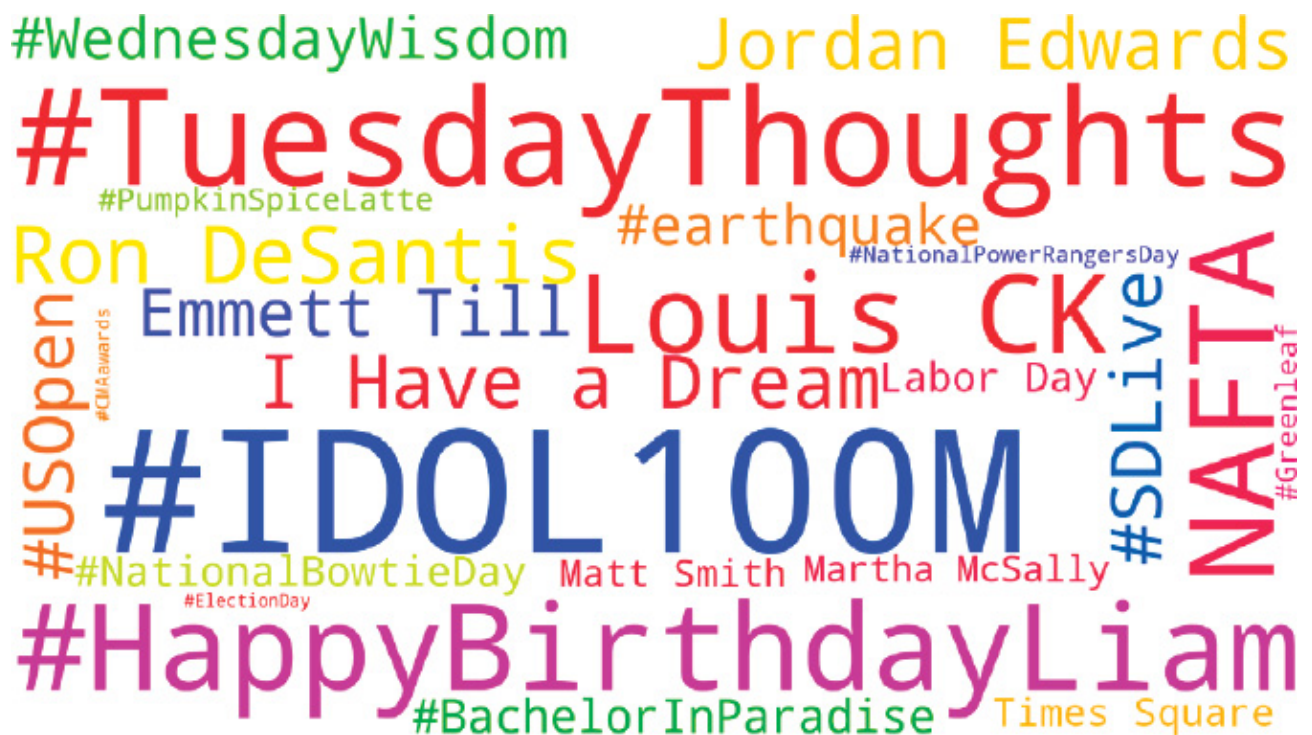
n [50]: wordcloud = WordCloud(width=1600, height=900,
...:     prefer_horizontal=0.5, min_font_size=10, colormap='prism',
...:     background_color='white')
...:

In [51]: wordcloud = wordcloud.fit_words(topics)

In [52]: wordcloud = wordcloud.to_file('TrendingTwitter.png')

```

The resulting word cloud is shown below—yours will differ based on the trending topics the day you run the code:



12.12 CLEANING/PREPROCESSING TWEETS FOR ANALYSIS

Data cleaning is one of the most common tasks that data scientists perform. Depending on how you intend to process tweets, you'll need to use natural language processing to normalize them by performing some or all of the data cleaning tasks in the following table. Many of these can be performed using the libraries introduced in the “Natural Language Processing (NLP)” chapter:

Tweet cleaning tasks	
Converting all text to the same case	Removing stop words

Removing # symbol from hashtags	Removing RT (retweet) and FAV (favorite)
Removing @-mentions	Removing URLs
Removing duplicates	Stemming
Removing excess whitespace	Lemmatization
Removing hashtags	Tokenization
Removing punctuation	

tweet-preprocessor Library and TextBlob Utility Functions

In this section, we'll use the **tweet-preprocessor library**

<https://github.com/s/preprocessor>

to perform some basic tweet cleaning. It can automatically remove any combination of:

- URLs,
- @-mentions (like @nasa),
- hashtags (like #mars),
- Twitter reserved words (like, RT for retweet and FAV for favorite, which is similar to a “like” on other social networks),
- emojis (all or just smileys) and
- numbers

The following table shows the module's constants representing each option:

Option	Option constant
@-Mentions (e.g., @nasa)	OPT.MENTION

Emoji	OPT.EMOJI
Hashtag (e.g., #mars)	OPT.HASHTAG
Number	OPT.NUMBER
Reserved Words (RT and FAV)	OPT.RESERVED
Smiley	OPT.SMILEY
URL	OPT.URL

Installing tweet-preprocessor

To install tweet-preprocessor, open your Anaconda Prompt (Windows), Terminal (macOS/Linux) or shell (Linux), then issue the following command:

```
pip install tweet-preprocessor
```

Windows users might need to run the Anaconda Prompt as an administrator for proper software installation privileges. To do so, right-click Anaconda Prompt in the start menu and select **More > Run as administrator**.

Cleaning a Tweet

Let's do some basic tweet cleaning that we'll use in a later example in this chapter. The tweet-preprocessor library's module name is `preprocessor`. Its documentation recommends that you import the module as follows:

[lick here to view code image](#)

```
In [1]: import preprocessor as p
```

To set the cleaning options you'd like to use call the module's `set_options` function. In this case, we'd like to remove URLs and Twitter reserved words:

[lick here to view code image](#)

```
In [2]: p.set_options(p.OPT.URL, p.OPT.RESERVED)
```

Now let's clean a sample tweet containing a reserved word (RT) and a URL:

[lick here to view code image](#)

```
In [3]: tweet_text = 'RT A sample retweet with a URL https://nasa.gov'

In [4]: p.clean(tweet_text)
Out[4]: 'A sample retweet with a URL'
```

12.13 TWITTER STREAMING API

Twitter's free Streaming API sends to your app *randomly selected* tweets dynamically as they occur—up to a maximum of one percent of the tweets per day. According to `InternetLiveStats.com`, there are approximately 6000 tweets per second, which is over 500 million tweets per day.³ So the Streaming API gives you access to approximately five million tweets per day. Twitter used to allow free access to 10% of streaming tweets, but this service—called the *fire hose*—is now available only as a paid service. In this section, we'll use a class definition and an IPython session to walk through the steps for processing streaming tweets. Note that the code for receiving a tweet stream requires creating a *custom class* that *inherits* from another class. These topics are covered in chapter 10.

³ <http://www.internetlivestats.com/twitter-statistics/>.

12.13.1 Creating a Subclass of `StreamListener`

The Streaming API returns tweets as they happen that match your search criteria. Rather than connecting to Twitter on each method call, a stream uses a *persistent* connection to **push** (that is, send) tweets to your app. The rate at which those tweets arrive varies tremendously, based on your search criteria. The more popular a topic is, the more likely it is that the tweets will arrive quickly.

You create a subclass of Tweepy's **`StreamListener` class** to process the tweet stream. An object of this class is the *listener* that's notified when each new tweet (or other

essage sent by Twitter ⁴) arrives. Each message Twitter sends results in a call to a `StreamListener` method. The following table summarizes several such methods. `StreamListener` already defines each method, so you redefine only the methods you need—this is known as *overriding*. For additional `StreamListener` methods, see:

⁴For details on the messages, see

<https://developer.twitter.com/en/docs/tweets/filter-realtime-guides/streaming-message-types.html>.

<https://github.com/tweepy/tweepy/blob/master/tweepy/streaming.py>

Method	Description
<code>on_connect(self)</code>	Called when you successfully connect to the Twitter stream. This is for statements that should execute only if your app is connected to the stream.
<code>on_status(self, status)</code>	Called when a tweet arrives— <code>status</code> is an object of Tweepy's <code>Status</code> .
<code>on_limit(self, track)</code>	Called when a limit notice arrives. This occurs if your search matches more tweets than Twitter can deliver based on its current streaming rate limits. In this case, the limit notice contains the number of matching tweets that could not be delivered.
<code>on_error(self, status_code)</code>	Called in response to error codes sent by Twitter.
<code>on_timeout(self)</code>	Called if the connection times out—that is, the Twitter server is not responding.
	Called if Twitter sends a disconnect warning to indicate

```
on_warning(self,  
notice)
```

that the connection might be closed. For example, Twitter maintains a queue of the tweets it's pushing to your app. If the app does not read the tweets fast enough, `on_warning`'s `notice` argument will contain a warning message indicating that the connection will terminate if the queue becomes full.

Class TweetListener

Our `StreamListener` subclass `TweetListener` is defined in `tweetlistener.py`. We discuss the `TweetListener`'s components here. Line 6 indicates that class `TweetListener` is a subclass of `tweepy.StreamListener`. This ensures that our new class has class `StreamListener`'s default method implementations.

[lick here to view code image](#)

```
1 # tweetlistener.py  
2 """tweepy.StreamListener subclass that processes tweets as they arrive.  
3 import tweepy  
4 from textblob import TextBlob  
5  
6 class TweetListener(tweepy.StreamListener):  
7     """Handles incoming Tweet stream."""  
8
```

Class TweetListener: __init__ Method

The following lines define the `TweetListener` class's `__init__` method, which is called when you create a new `TweetListener` object. The `api` parameter is the Tweepy API object that `TweetListener` will use to interact with Twitter. The `limit` parameter is the total number of tweets to process—10 by default. We added this parameter to enable you to control the number of tweets to receive. As you'll soon see, we terminate the stream when that `limit` is reached. If you set `limit` to `None`, the stream will not terminate automatically. Line 11 creates an instance variable to keep track of the number of tweets processed so far, and line 12 creates a constant to store the `limit`. If you're not familiar with `__init__` and `super()` from previous chapters, line 13 ensures that the `api` object is stored properly for use by your listener object.

[lick here to view code image](#)

```

9     def __init__(self, api, limit=10):
10         """Create instance variables for tracking number of tweets."""
11         self.tweet_count = 0
12         self.TWEET_LIMIT = limit
13         super().__init__(api) # call superclass's init
14

```

Class TweetListener: on_connect Method

Method `on_connect` is called when your app successfully connects to the Twitter stream. We override the default implementation to display a “Connection successful” message.

[lick here to view code image](#)

```

15     def on_connect(self):
16         """Called when your connection attempt is successful, enabling
17         you to perform appropriate application tasks at that point."""
18         print('Connection successful\n')
19

```

Class TweetListener: on_status Method

Method `on_status` is called by Tweepy when each tweet arrives. This method’s second parameter receives a Tweepy Status object representing the tweet. Lines 23–26 get the tweet’s text. First, we assume the tweet uses the new 280-character limit, so we attempt to access the tweet’s `extended_tweet` property and get its `full_text`. An exception will occur if the tweet does not have an `extended_tweet` property. In this case, we get the `text` property instead. Lines 28–30 then display the `screen_name` of the user who sent the tweet, the `lang` (that is language) of the tweet and the `tweet_text`. If the language is not English (`'en'`), lines 32–33 use a `TextBlob` to translate the tweet and display it in English. We increment `self.tweet_count` (line 36), then compare it to `self.TWEET_LIMIT` in the return statement. If `on_status` returns `True`, the stream remains open. When `on_status` returns `False`, Tweepy disconnects from the stream.

[lick here to view code image](#)

```

20     def on_status(self, status):
21         """Called when Twitter pushes a new tweet to you."""
22         # get the tweet text
23         try:
24             tweet_text = status.extended_tweet.full_text
25         except:
26             tweet_text = status.text

```

```

27
28     print(f'Screen name: {status.user.screen_name}:')
29     print(f'    Language: {status.lang}')
30     print(f'        Status: {tweet_text}')
31
32     if status.lang != 'en':
33         print(f' Translated: {TextBlob(tweet_text).translate()}')
34
35     print()
36     self.tweet_count     += 1   # track number of     tweets processed
37
38     # if TWEET_LIMIT is reached, return False to     terminate streamin
39     return self.tweet_count != self.TWEET_LIMIT

```

2.13.2 Initiating Stream Processing

Let's use an IPython session to test our new TweetListener.

Authenticating

First, you must authenticate with Twitter and create a Tweepy API object:

[lick here to view code image](#)

```

In [1]: import tweepy

In [2]: import keys

In [3]: auth = tweepy.OAuthHandler(keys.consumer_key,
...:                               keys.consumer_secret)
...:
...:

In [4]: auth.set_access_token(keys.access_token,
...:                          keys.access_token_secret)
...:
...:

In [5]: api = tweepy.API(auth, wait_on_rate_limit=True,
...:                          wait_on_rate_limit_notify=True)
...:
...:

```

Creating a TweetListener

Next, create an object of the TweetListener class and initialize it with the api object:

[lick here to view code image](#)

```

In [6]: from tweetlistener import TweetListener

```

```
In [7]: tweet_listener = TweetListener(api)
```

We did not specify the `limit` argument, so this `TweetListener` terminates after 10 tweets.

Creating a Stream

A Tweepy **Stream** object manages the connection to the Twitter stream and passes the messages to your `TweetListener`. The `Stream` constructor's `auth` keyword argument receives the `api` object's `auth` property, which contains the previously configured `OAuthHandler` object. The `listener` keyword argument receives your listener object:

[lick here to view code image](#)

```
In [8]: tweet_stream = tweepy.Stream(auth=api.auth,  
...:                                listener=tweet_listener)  
...:
```

Starting the Tweet Stream

The `Stream` object's **filter method** begins the streaming process. Let's track tweets about the NASA Mars rovers. Here, we use the `track` parameter to pass a list of search terms:

[lick here to view code image](#)

```
In [9]: tweet_stream.filter(track=['Mars Rover'], is_async=True)
```

The Streaming API will return full tweet JSON objects for tweets that match any of the terms, not just in the tweet's text, but also in @-mentions, hashtags, expanded URLs and other information that Twitter maintains in a tweet object's JSON. So, you might not see the search terms you're tracking if you look only at the tweet's text.

Asynchronous vs. Synchronous Streams

The `is_async=True` argument indicates that `filter` should initiate an **asynchronous tweet stream**. This allows your code to continue executing while your listener waits to receive tweets and is useful if you decide to terminate the stream early. When you execute an asynchronous tweet stream in IPython, you'll see the next `In []` prompt and can terminate the tweet stream by setting the `Stream` object's **running property** to `False`, as in:


```
tweet_stream.running=False
```

Without the `is_async=True` argument, `filter` initiates a **synchronous tweet stream**. In this case, IPython would display the next `In []` prompt *after* the stream terminates. Asynchronous streams are particularly handy in GUI applications so your users can continue to interact with other parts of the application while tweets arrive. The following shows a portion of the output consisting of two tweets:

[lick here to view code image](#)

```
Connection successful

Screen name: bevjoy:
  Language: en
    Status: RT @SPACEdotcom: With Mars Dust Storm   Clearing, Opportunity R

screen name: tourmalinel973:
  Language: en
    Status: RT @BennuBirdy: Our beloved Mars rover isn't   done yet, but sh

..
```

Other `filter` Method Parameters

Method `filter` also has parameters for refining your tweet searches by Twitter user ID numbers (to follow tweets from specific users) and by location. For details, see:

<https://developer.twitter.com/en/docs/tweets/filter-realtime/guides/basic-stream-arameters>

Twitter Restrictions Note

Marketers, researchers and others frequently store tweets they receive from the Streaming API. If you're storing tweets, Twitter requires you to delete any message or location data for which you receive a deletion message. This will occur if a user deletes a tweet or the tweets location data after Twitter pushes that tweet to you. In each case, your listener's **`on_delete` method** will be called. For deletion rules and message details, see

```
https://developer.twitter.com/en/docs/tweets/filter-realtime/guides/streamin
```

2.14 TWEET SENTIMENT ANALYSIS

In the “ Natural Language Processing (NLP)” chapter, we demonstrated sentiment analysis on sentences. Many researchers and companies perform sentiment analysis on tweets. For example, political researchers might check tweet sentiment during elections season to understand how people feel about specific politicians and issues. Companies might check tweet sentiment to see what people are saying about their products and competitors’ products.

In this section, we’ll use the techniques introduced in the preceding section to create a script (`sentimentlistener.py`) that enables you to check the sentiment on a specific topic. The script will keep totals of all the positive, neutral and negative tweets it processes and display the results.

The script receives two command-line arguments representing the topic of the tweets you wish to receive and the number of tweets for which to check the sentiment—only those tweets that are not eliminated are counted. For viral topics, there are large numbers of retweets, which we are not counting, so it could take some time get the number of tweets you specify. You can run the script from the `ch12` folder as follows:

[lick here to view code image](#)

```
ipython sentimentlistener.py football 10
```

which produces output like the following. Positive tweets are preceded by a +, negative tweets by a – and neutral tweets by a space:

[lick here to view code image](#)

```
ftblNeutral: Awful game of football. So boring slow   hoofball complete was
+ TBulmer28: I've seen 2 successful onside kicks within a   40 minute span. 1
+ CMayADay12: The last normal Sunday for the next couple   months. Don't text
rpimusic: My heart legitimately hurts for Kansas football   fans
+ DSCunningham30: @LeahShieldsWPSD It's awesome that u like   college football
damanr: I'm bummed I don't know enough about football to   roast @samesfanc
+ jamesianosborne: @TheRochaSays @WatfordFC @JackHind   Haha.... just when yo
+ Tshanerbeer: @PennStateFball @PennStateOnBTN Ah yes,   welcome back college
- cougarhokie: @hokiehack @skiptyler I can verify the   badness of that footk
```

```
+ Unite_Reddevils: @Pablo_di_Don Well make yourself clear it's football not  
  
Tweet sentiment for "football"  
Positive: 6  
Neutral: 2  
Negative: 2
```

he script (`sentimentlistener.py`) is presented below. We focus only on the new capabilities in this example.

Imports

Lines 4–8 import the `keys.py` file and the libraries used throughout the script:

[lick here to view code image](#)

```
1 # sentimentlisener.py  
2 """Script that searches for tweets that match a search string  
3 and tallies the number of positive, neutral and negative tweets."""  
4 import keys  
5 import preprocessor as p  
6 import sys  
7 from textblob import TextBlob  
8 import tweepy  
9
```

Class `SentimentListener`: `__init__` Method

In addition to the API object that interacts with Twitter, the `__init__` method receives three additional parameters:

- `sentiment_dict`—a dictionary in which we'll keep track of the tweet sentiments,
- `topic`—the topic we're searching for so we can ensure that it appears in the tweet text and
- `limit`—the number of tweets to process (not including the ones we eliminate).

Each of these is stored in the current `SentimentListener` object (`self`).

[lick here to view code image](#)

```
10 class SentimentListener(tweepy.StreamListener):  
11     """Handles incoming Tweet stream."""
```

```

12
13     def __init__(self, api, sentiment_dict, topic, limit=10):
14         """Configure the SentimentListener."""
15         self.sentiment_dict = sentiment_dict
16         self.tweet_count = 0
17         self.topic = topic
18         self.TWEET_LIMIT = limit
19
20         # set tweet-preprocessor to remove URLs/reserved words
21         p.set_options(p.OPT.URL, p.OPT.RESERVED)
22         super().__init__(api) # call superclass's init
23

```

Method on_status

When a tweet is received, method on_status:

- gets the tweet's text (lines 27–30)
- skips the tweet if it's a retweet (lines 33–34)
- cleans the tweet to remove URLs and reserved words like RT and FAV (line 36)
- skips the tweet if it does not have the topic in the tweet text (lines 39–40)
- uses a TextBlob to check the tweet's sentiment and updates the sentiment_dict accordingly (lines 43–52)
- prints the tweet text (line 55) preceded by + for positive sentiment, space for neutral sentiment or – for negative sentiment and
- checks whether we've processed the specified number of tweets yet (lines 57–60).

[lick here to view code image](#)

```

24     def on_status(self, status):
25         """Called when Twitter pushes a new tweet to you."""
26         # get the tweet's text
27         try:
28             tweet_text = status.extended_tweet.full_text
29         except:
30             tweet_text = status.text
31
32         # ignore retweets
33         if tweet_text.startswith('RT'):
34             return
35
36         tweet_text = p.clean(tweet_text) # clean the tweet

```

```

37
38     # ignore tweet if the topic is not in the tweet    text
39     if self.topic.lower() not in    tweet_text.lower():
40         return
41
42     # update self.sentiment_dict with the polarity
43     blob =    TextBlob(tweet_text)
44     if blob.sentiment.polarity > 0:
45         sentiment    = '+'
46         self.sentiment_dict['positive'] += 1
47     elif blob.sentiment.polarity == 0:
48         sentiment    = ' '
49         self.sentiment_dict['neutral'] += 1
50     else:
51         sentiment    = '-'
52         self.sentiment_dict['negative'] += 1
53
54     # display the tweet
55     print(f'{sentiment} {status.user.screen_name}: {tweet_text}\n')
56
57     self.tweet_count    += 1    # track number of tweets    processed
58
59     # if TWEET_LIMIT is reached, return False to    terminate streamin
60     return self.tweet_count != self.TWEET_LIMIT
61

```

ain Application

The main application is defined in the function `main` (lines 62–87; discussed after the following code), which is called by lines 90–91 when you execute the file as a script. So `sentiment-listener.py` can be imported into IPython or other modules to use class `SentimentListener` as we did with `TweetListener` in the previous section:

[lick here to view code image](#)

```

62 def main():
63     # configure the OAuthHandler
64     auth =    tweepy.OAuthHandler(keys.consumer_key, keys.consumer_secret)
65     auth.set_access_token(keys.access_token,    keys.access_token_secret)
66
67     # get the API object
68     api =    tweepy.API(auth, wait_on_rate_limit=True,
69                             wait_on_rate_limit_notify=True)
70
71     # create the StreamListener subclass object
72     search_key    = sys.argv[1]
73     limit =    int(sys.argv[2])    #    number of tweets to tally
74     sentiment_dict    = {'positive': 0, 'neutral': 0, 'negative': 0}
75     sentiment_listener    = SentimentListener(api,
76         sentiment_dict,    search_key, limit)

```

```

77
78     # set up Stream
79     stream = tweepy.Stream(auth=api.auth, listener=sentiment_listener)
80
81     # start filtering English tweets containing search_key
82     stream.filter(track=[search_key], languages=['en'], is_async=False)
83
84     print(f'Tweet sentiment for "{search_key}"')
85     print('Positive:', sentiment_dict['positive'])
86     print('Neutral:', sentiment_dict['neutral'])
87     print('Negative:', sentiment_dict['negative'])
88
89 # call main if this file is executed as a script
90 if __name__ == '__main__':
91     main()

```

lines 72–73 get the command-line arguments. Line 74 creates the `sentiment_dict` dictionary that keeps track of the tweet sentiments. Lines 75–76 create the `SentimentListener`. Line 79 creates the `Stream` object. We once again initiate the stream by calling `Stream` method `filter` (line 82). However, this example uses a synchronous stream so that lines 84–87 display the sentiment report only after the specified number of tweets (`limit`) are processed. In this call to `filter`, we also provided the keyword argument `languages`, which specifies a list of language codes. The one language code `'en'` indicates Twitter should return only English language tweets.

12.15 GEOCODING AND MAPPING

In this section, we'll collect streaming tweets, then plot the locations of those tweets. Most tweets do not include latitude and longitude coordinates, because Twitter disables this by default for all users. Those who wish to include their precise location in tweets must opt into that feature. Though most tweets do not include precise location information, a large percentage include the user's home location information; however, even that is sometimes invalid, such as "Far Away" or a fictitious location from a user's favorite movie.

In this section, for simplicity, we'll use the `location` property of the tweet's `User` object to plot that user's location on an interactive map. The map will let you zoom in and out and drag to move the map around so you can look at different areas (known as *panning*). For each tweet, we'll display a map marker that you can click to see a popup containing the user's screen name and tweet text.

We'll ignore retweets and tweets that do not contain the search topic. For other tweets, we'll track the percentage of tweets with location information. When we get the latitude and longitude information for those locations, we'll also track the percentage of those tweets that had invalid location data.

geopy Library

We'll use the **geopy library** (<https://github.com/geopy/geopy>) to translate locations into latitude and longitude coordinates—known as **geocoding**—so we can place markers on a map. The library supports dozens of geocoding web services, many of which have free or lite tiers. For this example, we'll use the **OpenMapQuest geocoding service** (discussed shortly). You installed geopy in [section 12.6](#).

OpenMapQuest Geocoding API

We'll use the OpenMapQuest Geocoding API to convert locations, such as Boston, MA into their latitudes and longitudes, such as 42.3602534 and -71.0582912, for plotting on maps. OpenMapQuest currently allows 15,000 transactions per month on their free tier. To use the service, first sign up at

<https://developer.mapquest.com/>

Once logged in, go to

```
https://developer.mapquest.com/user/me/apps
```

and click **Create a New Key**, fill in the **App Name** field with a name of your choosing, leave the **Callback URL** empty and click **Create App** to create an API key. Next, click your app's name in the web page to see your consumer key. In the `keys.py` file you used earlier in the chapter, store the consumer key by replacing *YourKeyHere* in the line

```
mapquest_key = 'YourKeyHere'
```

As we did earlier in the chapter, we'll import `keys.py` to access this key.

Folium Library and Leaflet.js JavaScript Mapping Library

For the maps in this example, we'll use the **folium library**

<https://github.com/python-visualization/folium>

which uses the popular Leaflet.js JavaScript mapping library to display maps. The maps that folium produces are saved as HTML files that you can view in your web browser. To install folium, execute the following command:

```
pip install folium
```

Maps from OpenStreetMap.org

By default, Leaflet.js uses open source maps from `OpenStreetMap.org`. These maps are copyrighted by the OpenStreetMap.org contributors. To use these maps ⁵, they require the following copyright notice:

⁵ https://wiki.osmfoundation.org/wiki/Licence/Licence_and_Legal_FAQ.

[lick here to view code image](#)

```
Map data © OpenStreetMap contributors
```

and they state:

You must make it clear that the data is available under the Open Database License. This can be achieved by providing a “License” or “Terms” link which links to

www.openstreetmap.org/copyright or

www.opendatacommons.org/licenses/odbl.

12.15.1 Getting and Mapping the Tweets

Let’s interactively develop the code that plots tweet locations. We’ll use utility functions from our `tweetutilities.py` file and class `LocationListener` in `locationlistener.py`. We’ll explain the details of the utility functions and class in the subsequent sections.

Get the API Object

As in the other streaming examples, let’s authenticate with Twitter and get the Tweepy API object. In this case, we do this via the `get_API` utility function in `tweetutilities.py`:

[lick here to view code image](#)

```
In [1]: from tweetutilities import get_API

In [2]: api = get_API()
```

Collections Required By `LocationListener`

Our `LocationListener` class requires two collections: A list (`tweets`) to store the tweets we collect and a dictionary (`counts`) to track the total number of tweets we collect and the number that have location data:

[lick here to view code image](#)

```
In [3]: tweets = []

In [4]: counts = {'total_tweets': 0, 'locations': 0}
```

Creating the LocationListener

For this example, the `LocationListener` will collect 50 tweets about 'football':

[lick here to view code image](#)

```
In [5]: from locationlistener import LocationListener

In [6]: location_listener = LocationListener(api, counts_dict=counts,
...:     tweets_list=tweets, topic='football', limit=50)
...:
```

The `LocationListener` will use our utility function `get_tweet_content` to extract the screen name, tweet text and location from each tweet, place that data in a dictionary.

Configure and Start the Stream of Tweets

Next, let's set up our `Stream` to look for English language 'football' tweets:

[lick here to view code image](#)

```
In [7]: import tweepy

In [8]: stream = tweepy.Stream(auth=api.auth, listener=location_listener)

In [9]: stream.filter(track=['football'], languages=['en'], is_async=False)
```

Now wait to receive the tweets. Though we do not show them here (to save space), the `LocationListener` displays each tweet's screen name and text so you can see the live stream. If you're not receiving any (perhaps because it is not football season), you might want to type `Ctrl + C` to terminate the previous snippet then try again with a different search term.

Displaying the Location Statistics

When the next `In []` prompt displays, we can check how many tweets we processed, how many had locations and the percentage that had locations:

[lick here to view code image](#)

```
In [10]: counts['total_tweets']
Out[10]: 63

In [11]: counts['locations']
Out[11]: 50

In [12]: print(f'{counts["locations"] / counts["total_tweets"]:.1%}')
79.4%
```

In this particular execution, 79.4% of the tweets contained location data.

Geocoding the Locations

Now, let's use our `get_geocodes` utility function from `tweetutilities.py` to geocode the location of each tweet stored in the list `tweets`:

[lick here to view code image](#)

```
In [13]: from tweetutilities import get_geocodes

In [14]: bad_locations = get_geocodes(tweets)
Getting coordinates for tweet locations...
OpenMapQuest service timed out. Waiting.
OpenMapQuest service timed out. Waiting.
Done geocoding
```

Sometimes the OpenMapQuest geocoding service times out, meaning that it cannot handle your request immediately and you need to try again. In that case, our function `get_geocodes` displays a message, waits for a short time, then retries the geocoding request.

As you'll soon see, for each tweet with a *valid* location, the `get_geocodes` function adds to the tweet's dictionary in the `tweets` list two new keys—`'latitude'` and `'longitude'`. For the corresponding values, the function uses the tweet's coordinates that OpenMapQuest returns.

Displaying the Bad Location Statistics

When the next `In []` prompt displays, we can check the percentage of tweets that had invalid location data:

[lick here to view code image](#)

```
In [15]: bad_locations
Out[15]: 7

In [16]: print(f'{bad_locations    / counts["locations"]:.1%}')
14.0%
```

In this case, of the 50 tweets with location data, 7 (14%) had invalid locations.

Cleaning the Data

Before we plot the tweet locations on a map, let's use a pandas `DataFrame` to clean the data. When you create a `DataFrame` from the `tweets` list, it will contain the value `NaN` for the `'latitude'` and `'longitude'` of any tweet that did not have a valid location. We can remove any such rows by calling the `DataFrame`'s **dropna method**:

[lick here to view code image](#)

```
In [17]: import pandas as pd

In [18]: df = pd.DataFrame(tweets)

In [19]: df = df.dropna()
```

Creating a Map with Folium

Now, let's create a folium **Map** on which we'll plot the tweet locations:

[lick here to view code image](#)

```
In [20]: import folium

In [21]: usmap = folium.Map(location=[39.8283, -98.5795],
...:                               tiles='Stamen Terrain',
...:                               zoom_start=5, detect_retina=True)
...:
```

The `location` keyword argument specifies a sequence containing latitude and longitude coordinates for the map's center point. The values above are the geographic center of the continental United States (<http://bit.ly/CenterOfTheUS>). It's possible that some of the tweets we plot will be outside the U.S. In this case, you will not see them initially when you open the map. You can zoom in and out using the `+` and `-` buttons at the top-left of the map, or you can pan the map by dragging it with the mouse to see anywhere in the world.

The `zoom_start` keyword argument specifies the map's initial zoom level, lower values show more of the world and higher values show less. On our system, 5 displays the entire continental United States. The `detect_retina` keyword argument enables folium to detect high-resolution screens. When it does, it requests higher-resolution maps from OpenStreetMap.org and changes the zoom level accordingly.

Creating Popup Markers for the Tweet Locations

Next, let's iterate through the `DataFrame` and add to the Map folium `Popup` objects containing each tweet's text. In this case, we'll use method `itertuples` to create tuples from each row of the `DataFrame`. Each tuple will contain a property for each `DataFrame` column:

[lick here to view code image](#)

```
In [22]: for t in df.itertuples():
...:     text = ': '.join([t.screen_name, t.text])
...:     popup = folium.Popup(text, parse_html=True)
...:     marker = folium.Marker((t.latitude, t.longitude),
...:                             popup=popup)
...:     marker.add_to(usmap)
...:
```

First, we create a string (`text`) containing the user's `screen_name` and tweet `text` separated by a colon. This will be displayed on the map if you click the corresponding marker. The second statement creates a folium `Popup` to display the text. The third statement creates a folium `Marker` object using a tuple to specify the `Marker`'s latitude and longitude. The `popup` keyword argument associates the tweet's `Popup` object with the new `Marker`. Finally, the last statement calls the `Marker`'s `add_to` method to specify the Map that will display the `Marker`.

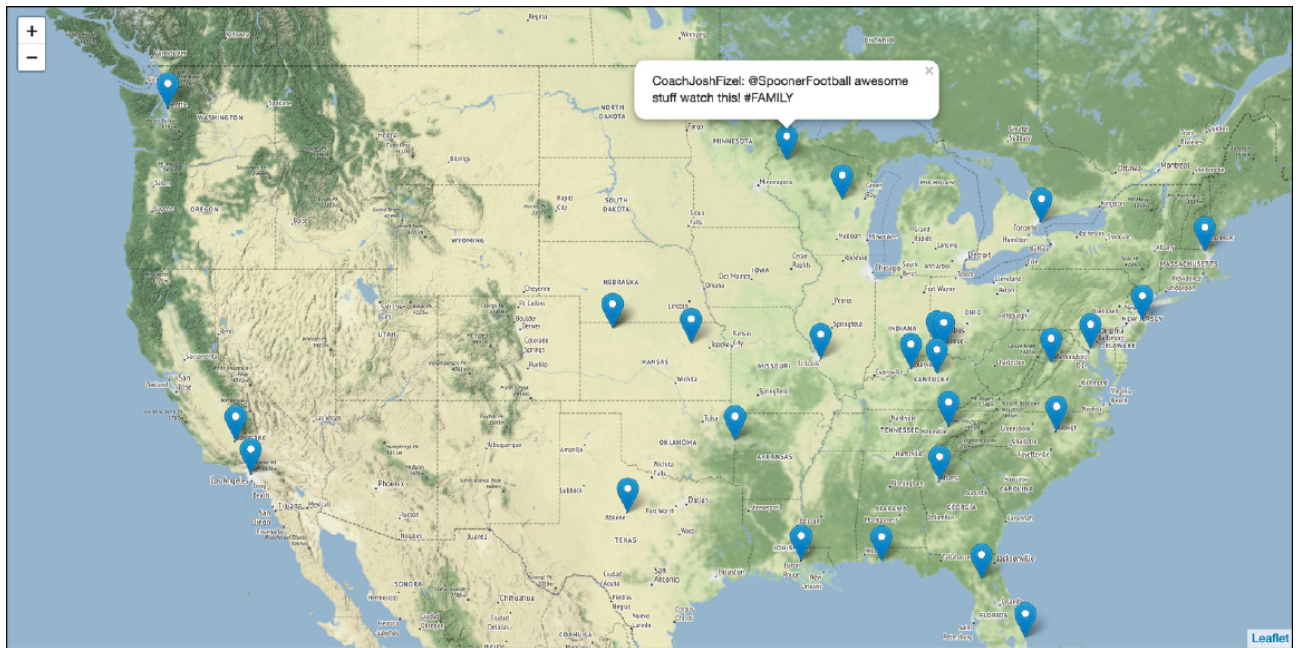
Saving the Map

The last step is to call the Map's `save` method to store the map in an HTML file, which you can then double click to open in your web browser:

[lick here to view code image](#)

```
In [23]: usmap.save('tweet_map.html')
```

The resulting map follows. The `Markers` on your map will differ:



Map data © OpenStreetMap contributors.

The data is available under the Open Database License

<http://www.openstreetmap.org/copyright>.

12.15.2 Utility Functions in `tweetutilities.py`

Here we present the utility functions `get_tweet_content` and `get_geo_codes` used in the preceding section's IPython session. In each case, the line numbers start from 1 for discussion purposes. These are both defined in `tweetutilities.py`, which is included in the `ch12 examples` folder.

`get_tweet_content` Utility Function

Function `get_tweet_content` receives a `Status` object (`tweet`) and creates a dictionary containing the tweet's `screen_name` (line 4), `text` (lines 7–10) and `location` (lines 12–13). The location is included only if the `location` keyword argument is `True`. For the tweet's text, we try to use the `full_text` property of an `extended_tweet`. If it's not available, we use the `text` property:

[lick here to view code image](#)

```
1 def get_tweet_content(tweet, location=False):
2     """Return dictionary with data from tweet    (a Status object)."""
3     fields = {}
4     fields['screen_name'] = tweet.user.screen_name
5
6     # get the tweet's text
7     try:
8         fields['text'] = tweet.extended_tweet.full_text
9     except:
10        fields['text'] = tweet.text
```

```

11
12     if location:
13         fields['location'] = tweet.user.location
14
15     return fields

```

get_geocodes Utility Function

Function `get_geocodes` receives a list of dictionaries containing tweets and geocodes their locations. If geocoding is successful for a tweet, the function adds the latitude and longitude to the tweet's dictionary in `tweet_list`. This code requires class **OpenMapQuest** from the `geopy` module, which we import into the file `tweetutilities.py` as follows:

```
from geopy import OpenMapQuest
```

[lick here to view code image](#)

```

1 def get_geocodes(tweet_list):
2     """Get the latitude and longitude for each tweet's location.
3     Returns the number of tweets with invalid location data."""
4     print('Getting coordinates for tweet locations...')
5     geo = OpenMapQuest(api_key=keys.mapquest_key) # geocoder
6     bad_locations = 0
7
8     for tweet in tweet_list:
9         processed = False
10        delay = .1 # used if OpenMapQuest times out to delay next c
11        while not processed:
12            try: # get coordinates for tweet['location']
13                geo_location = geo.geocode(tweet['location'])
14                processed = True
15            except: # timed out, so wait before trying again
16                print('OpenMapQuest service timed out. Waiting.')
17                time.sleep(delay)
18                delay += .1
19
20        if geo_location:
21            tweet['latitude'] = geo_location.latitude
22            tweet['longitude'] = geo_location.longitude
23        else:
24            bad_locations += 1 # tweet['location'] was invalid
25
26    print('Done geocoding')
27    return bad_locations

```

The function operates as follows:

- Line 5 creates the `OpenMapQuest` object we'll use to geocode locations. The `api_key` keyword argument is loaded from the `keys.py` file you edited earlier.
- Line 6 initializes `bad_locations` which we use to keep track of the number of invalid locations in the tweet objects we collected.
- In the loop, lines 9–18 attempt to geocode the current tweet's location. Sometimes the OpenMapQuest geocoding service will time out, meaning that it's temporarily unavailable. This can happen if you make too many requests too quickly. So, the `while` loop continues executing as long as `processed` is `False`. In each iteration, this loop calls the `OpenMapQuest` object's **geocode method** with the tweet's location string as an argument. If successful, `processed` is set to `True` and the loop terminates. Otherwise, lines 16–18 display a time-out message, wait for `delay` seconds and increase the delay in case we get another time out. Line 17 calls the Python Standard Library `time` module's `sleep` method to pause the code execution.
- After the `while` loop terminates, lines 20–24 check whether location data was returned and, if so, add it to the tweet's dictionary. Otherwise, line 24 increments the `bad_locations` counter.
- Finally, the function prints a message that it's done geocoding and returns the `bad_locations` value.

12.15.3 Class `LocationListener`

Class `LocationListener` performs many of the same tasks we demonstrated in the prior streaming examples, so we'll focus on just a few lines in this class:

[lick here to view code image](#)

```

1 # locationlistener.py
2 """Receives tweets matching a search string and stores a list of
3 dictionaries containing each tweet's screen_name/text/location."""
4 import tweepy
5 from tweetutilities import get_tweet_content
6
7 class LocationListener(tweepy.StreamListener):
8     """Handles incoming Tweet stream to get location data."""
9
10     def __init__(self, api, counts_dict, tweets_list, topic, limit=10):
11         """Configure the LocationListener."""
12         self.tweets_list = tweets_list
13         self.counts_dict = counts_dict
14         self.topic = topic
15         self.TWEET_LIMIT = limit

```

```

16         super().__init__(api)      # call superclass's init
17
18     def on_status(self, status):
19         """Called when Twitter pushes a new tweet to you."""
20         # get each tweet's screen_name, text and location
21         tweet_data = get_tweet_content(status, location=True)
22
23         # ignore retweets and tweets that do not contain the topic
24         if (tweet_data['text'].startswith('RT') or
25             self.topic.lower() not in tweet_data['text'].lower()):
26             return
27
28         self.counts_dict['total_tweets'] += 1 # original tweet
29
30         # ignore tweets with no location
31         if not status.user.location:
32             return
33
34         self.counts_dict['locations'] += 1 # tweet with location
35         self.tweets_list.append(tweet_data) # store the tweet
36         print(f'{status.user.screen_name}: {tweet_data["text"]}\n')
37
38         # if TWEET_LIMIT is reached, return False to terminate streaming
39         return self.counts_dict['locations'] != self.TWEET_LIMIT

```

In this case, the `__init__` method receives a `counts` dictionary that we use to keep track of the total number of tweets processed and a `tweet_list` in which we store the dictionaries returned by the `get_tweet_content` utility function.

Method `on_status`:

- Calls `get_tweet_content` to get the screen name, text and location of each tweet.
- Ignores the tweet if it is a retweet or if the text does not include the topic we're searching for—we'll use only original tweets containing the search string.
- Adds 1 to the value of the `'total_tweets'` key in the `counts` dictionary to track the number of original tweets we process.
- Ignores tweets that have no location data.
- Adds 1 to the value of the `'locations'` key in the `counts` dictionary to indicate that we found a tweet with a location.
- Appends to the `tweets_list` the `tweet_data` dictionary that `get_tweet_content` returned.

- Displays the tweet’s screen name and tweet text so you can see that the app is making progress.
- Checks whether the `TWEET_LIMIT` has been reached and, if so, returns `False` to terminate the stream.

12.16 WAYS TO STORE TWEETS

For analysis, you’ll commonly store tweets in:

- CSV files—A file format that we introduced in the “Files and Exceptions” chapter.
- pandas `DataFrames` in memory—CSV files can be loaded easily into `DataFrames` for cleaning and manipulation.
- SQL databases—Such as MySQL, a free and open source relational database management system (RDBMS).
- NoSQL databases—Twitter returns tweets as JSON documents, so the natural way to store them is in a NoSQL JSON document database, such as MongoDB. Tweepy generally hides the JSON from the developer. If you’d like to manipulate the JSON directly, use the techniques we present in the “Big Data: Hadoop, Spark, NoSQL and IoT” chapter, where we’ll look at the PyMongo library.

12.17 TWITTER AND TIME SERIES

A time series is a sequence of values with timestamps. Some examples are daily closing stock prices, daily high temperatures at a given location, monthly U.S. job-creation numbers, quarterly earnings for a given company and more. Tweets are natural for time-series analysis because they’re time stamped. In the “Machine Learning” chapter, we’ll use a technique called simple linear regression to make predictions with time series. We’ll take another look at time series in the “Deep Learning” chapter when we discuss recurrent neural networks.

12.18 WRAP-UP

In this chapter, we explored data mining Twitter, perhaps the most open and accessible of all the social media sites, and one of the most commonly used big-data sources. You created a Twitter developer account and connected to Twitter using your account credentials. We discussed Twitter’s rate limits and some additional rules, and the importance of conforming to them.

e looked at the JSON representation of a tweet. We used Tweepy—one of the most widely used Twitter API clients—to authenticate with Twitter and access its APIs. We saw that tweets returned by the Twitter APIs contain much metadata in addition to a tweet's text. We determined an account's followers and whom an account follows, and looked at a user's recent tweets.

We used Tweepy `Cursors` to conveniently request successive pages of results from various Twitter APIs. We used Twitter's Search API to download past tweets that met specified criteria. We used Twitter's Streaming API to tap into the flow of live tweets as they happened. We used the Twitter Trends API to determine trending topics for various locations and created a word cloud from trending topics.

We used the tweet-preprocessor library to clean and preprocess tweets to prepare them for analysis, and performed sentiment analysis on tweets. We used the folium library to create a map of tweet locations and interacted with it to see the tweets at particular locations. We enumerated common ways to store tweets and noted that tweets are a natural form of time series data. In the next chapter, we'll present IBM's Watson and its cognitive computing capabilities.